PlanCipher: aplicación cifrada para consultar planning, notas y asignaturas pendientes

Práctica final de Criptografía y Seguridad Informática (UC3M)

Esqueleto de la memoria. Los textos están redactados a partir del código entregado; revísalos con tus palabras (la defensa puede incluir preguntas individuales) y sustituye los campos [completar].

Límite: 13 páginas sin portada ni anexos, letra 11, interlineado simple, Calibri, Arial o Times New Roman.

Portada / datos del entregable

- Grupo: [completar]
- ID de grupo de prácticas: [completar]
- Nombre de todos los alumnos: [completar]
- Correo de todos los alumnos: [completar]
- Repositorio de código (enlace), si lo hubiera: [completar]

Reparto orientativo de páginas

Apartado	Páginas
1. Propósito y estructura	1
2. Autenticación de usuarios	2
3. Cifrado y gestión de claves, IVs y nonces	3
4. Integridad y autenticidad	1,5
5. Firma digital	2
6. Pruebas y ataques	2,5
7. Librerías e instalación	1

1. Propósito de la aplicación y estructura interna

Propósito. PlanCipher permite a los estudiantes consultar su planning, sus notas y las asignaturas que les quedan por aprobar, protegiendo la confidencialidad y la autenticidad de los datos. Hay dos roles: el estudiante, que gestiona su planning y consulta sus notas, y la secretaría, que matricula, publica el plan de estudios y emite las notas.

Qué datos protege y por qué. El planning es privado del estudiante y se cifra de extremo a extremo. Las notas son confidenciales y además deben ser auténticas: el estudiante necesita comprobar que las emitió la secretaría y que nadie las modificó. El plan de estudios es público, pero debe ser íntegro y auténtico.

Estructura interna. La aplicación es una CLI en Python con la lógica separada de las operaciones criptográficas:

- `plancipher/crypto/`: solo criptografía (`kdf`, `aead`, `hybrid`, `signing`, `keystore`, `certs`, `encoding`).
- `plancipher/`: lógica de aplicación (`auth`, `planning`, `academic`, `secretaria`, `storage`).
- `plancipher/cli.py`: solo entrada y salida.
- `tests/`: pruebas positivas y negativas con `unittest`.

Captura sugerida: árbol del proyecto y salida de `python -m plancipher --help`.

Diagrama sugerido: flujo secretaría → estudiante (matrícula firmada → nota cifrada y firmada → verificación y descifrado).

2. Autenticación de usuarios

Mecanismo. Autenticación basada en algo que se sabe (contraseña), que es la opción mínima obligatoria del enunciado. Cada comando del estudiante pide la contraseña sin eco; no se guarda ninguna sesión en disco.

Almacenamiento de contraseñas.

- scrypt con n=2^17, r=8, p=1 (unos 128 MiB de memoria) y una sal aleatoria de 128 bits por usuario, generada con `os.urandom`. [Justifica: función diseñada para ser costosa en memoria, resistente a GPU y ASIC.]
- Antes de derivar, la contraseña se normaliza a Unicode NFKC.
- Una sola ejecución de scrypt produce una clave maestra. HKDF-SHA256 con dos etiquetas distintas separa el **verificador de login**, que sí se guarda, de la **KEK** (clave de cifrado de claves), que nunca se guarda.
- La comparación del verificador se hace en tiempo constante (`hmac.compare_digest`).

- Los parámetros de scrypt se guardan junto a cada usuario, de modo que se pueden endurecer sin romper cuentas existentes.

Política y protecciones.

- Contraseña de al menos 12 caracteres, que no contenga el nombre de usuario y que no sea trivialmente repetitiva.
- Nombre de usuario validado con expresión regular; los nombres reservados (`secretaria`, `admin`, …) no se pueden registrar. El rol de secretaría no se obtiene por el nombre, sino por poseer su clave de firma.
- Bloqueo progresivo: tras 5 fallos, 30 s que se duplican hasta un máximo de 15 minutos.
- Mensaje de error genérico ("Usuario o contraseña incorrectos") y, para usuarios inexistentes, se ejecuta igualmente scrypt para que el tiempo de respuesta no delate si el usuario existe.

Por qué scrypt y no PBKDF2 o un hash simple. [Completar con tus palabras: coste en memoria, ataques con hardware especializado.]

Capturas sugeridas: registro correcto, contraseña rechazada por política, login incorrecto, bloqueo tras fallos, y el fichero `users.json` mostrando que solo hay sal, verificador y clave cifrada.

3. Cifrado simétrico y asimétrico: algoritmos, claves, IVs y nonces

Para qué se usa cada uno.

- *Simétrico (AES-256-GCM):* cifra el planning y el contenido de cada nota. Es cifrado autenticado (AEAD), como pide el enunciado siempre que sea posible.
- *Asimétrico (X25519 en esquema ECIES):* permite a la secretaría entregar la clave de contenido de una nota al estudiante usando solo su clave pública, sin conocer su contraseña.
- *Ambos:* las notas usan cifrado híbrido (datos con AES-GCM, clave de contenido con ECIES).

Parámetros.

- AES-256-GCM: clave de 256 bits, nonce de 96 bits, tag de 128 bits, con AAD.
- ECIES: par X25519 efímero por mensaje, secreto compartido derivado con HKDF-SHA256 (usando como sal la clave efímera y la del destinatario) y cifrado con AES-256-GCM. Se rechazan claves efímeras de orden bajo.

Generación y gestión de claves.

- Todas las claves, sales y nonces salen de `os.urandom` (CSPRNG del sistema).

- *Planning:* cada entrada tiene una clave de contenido (CEK) aleatoria de 256 bits, cifrada con la KEK del usuario. La KEK nunca se guarda: se deriva de la contraseña en cada login.
- *Clave privada X25519 del estudiante:* se guarda cifrada con la KEK (AAD = nombre de usuario).
- *Notas:* CEK aleatoria por nota, sellada para la clave pública del estudiante.
- Las claves no se imprimen nunca, ni siquiera con `--debug`.

Cómo se evita la reutilización de nonces. Cada cifrado genera un nonce aleatorio de 96 bits. Cada registro usa su propia CEK, y la KEK cifra muy pocos mensajes, así que quedamos muy por debajo del límite de 2^32 cifrados por clave con nonces aleatorios que recomienda NIST. Existe una prueba que genera 2000 cifrados y comprueba que no se repite ningún nonce.

AAD. El identificador y el propietario (o destinatario) viajan como datos autenticados adicionales, de modo que copiar un cifrado a otro usuario o a otro id invalida el tag.

Capturas sugeridas: `python -m plancipher --debug grades alice` (muestra algoritmo y longitud de clave) y `planning.json` con los datos cifrados.

4. Integridad y autenticidad de los datos

Qué se usa. Cifrado autenticado (AES-GCM) en los datos privados y firma digital en los datos que deben ser verificables por terceros. No se añade un HMAC aparte porque el tag de GCM ya autentica el contenido y el AAD; [justifícalo con tus palabras].

Gestión de claves. La clave de autenticación es la propia CEK de AES-GCM, gestionada como se explica en el apartado 3.

Qué ocurre cuando la verificación falla. El dato se rechaza y no se continúa el procesamiento como si fuera válido:

- Se registra un aviso con el algoritmo y el motivo.
- Se informa al usuario con `RECHAZADO` y el identificador corto del registro.
- La CLI devuelve el código de salida 3.
- El resto de registros válidos se siguen mostrando.

Capturas sugeridas: la demo con la nota o la entrada de planning modificada (`bash scripts/demo.sh`, apartados "ATAQUE 2" y "ATAQUE 3").

5. Firma digital

Para qué se usa. La secretaría firma tres tipos de información:

- *Matrícula:* la pareja (usuario, clave pública X25519). Así la secretaría sabe que cifra para la clave auténtica del estudiante y no para una sustituida por un atacante.
- *Plan de estudios:* integridad y autenticidad de un documento público.
- *Notas:* la firma cubre todo el sobre (cabecera con estudiante, número de secuencia y fecha, clave sellada y cifrado).

Algoritmo y parámetros. ECDSA con la curva P-256 y SHA-256. [Justifica: curva estándar actual, claves de 256 bits con seguridad equivalente a unos 128 bits, firmas compactas.] El mensaje firmado lleva un prefijo de dominio distinto para cada propósito (`enroll`, `plan`, `record`), por lo que una firma de un tipo no puede reutilizarse como otro.

Generación, almacenamiento y protección de claves.

- La clave privada de firma de la secretaría se genera al inicializar (no viene en el repositorio) y se guarda cifrada con una passphrase mediante scrypt y AES-256-GCM.
- El certificado X.509 autofirmado es el ancla de confianza. Se valida su fecha, su autofirma, el tipo de clave y, opcionalmente, una huella SHA-256 fijada con `--trust-fingerprint`.
- Al desbloquear la clave se comprueba que corresponde al certificado.

Verificación. Es automática: el estudiante verifica la firma antes de descifrar cada nota y el resultado queda en el log y en pantalla (`[firma verificada]`).

Prueba de rechazo (obligatoria). Muestra una nota con la firma o el cifrado modificado y la salida `RECHAZADO ... Firma digital inválida`. *Captura sugerida:* apartado "ATAQUE 2" de la demo.

Limitación (mencionarla suma en la defensa). El ancla de confianza es autofirmada; en un entorno real se usaría una PKI con CA raíz, CA intermedia y revocación.

6. Pruebas realizadas y ataques mitigados

Pruebas. 68 pruebas automáticas con `unittest` (`python -m unittest discover -s tests -t .`), repartidas en:

- *Cripto:* AEAD, cifrado híbrido, firma, derivación de claves, almacén de claves, certificados y codificación.
- *Autenticación:* registro, login, política de contraseñas, nombres reservados, bloqueo y manipulación del fichero de usuarios.
- *Planning y notas:* ida y vuelta, cifrado en disco, aislamiento entre usuarios y manipulaciones.
- *CLI de extremo a extremo* con parámetros scrypt reales.

Se usan parámetros scrypt reducidos solo en los tests para que sean rápidos; el programa usa los valores reales por defecto.

Pruebas positivas. Registro y login correctos, cifrado y descifrado, verificación de firma, nota vigente tras una recuperación, cálculo de asignaturas pendientes.

Pruebas negativas. Modificación de nonce, ciphertext o tag; AAD o clave incorrectos; firma alterada o de otra clave; certificado caducado, no autofirmado o con huella distinta; contraseña o passphrase incorrecta; registros modificados, reetiquetados, repetidos o antiguos.

Al menos tres errores o ataques relevantes y su mitigación. Elige y desarrolla los que quieras defender; el código cubre estos siete:

1. *Manipulación de datos en disco* (cifrado, tag, firma, cabecera): AEAD y ECDSA provocan el rechazo.
2. *Cambio de contexto* (copiar un registro a otro usuario o id): el AAD y la firma cubren propietario e identificador.
3. *Repetición y rollback de notas:* id único, número de secuencia creciente por estudiante y nota vigente igual a la de mayor secuencia.
4. *Sustitución de la clave pública del estudiante:* la matrícula firmada liga usuario y clave; una clave sustituida se detecta al emitir.
5. *Fuerza bruta de contraseñas:* scrypt costoso, sal única y bloqueo progresivo.
6. *Escalada de privilegios:* la secretaría no es un nombre de usuario; se necesita su clave de firma.
7. *Secretaría falsa:* las firmas se verifican contra el certificado fijado.

Limitaciones conocidas.

- Un atacante con escritura en disco puede borrar el último registro sin que se detecte; se resolvería con un contador o manifiesto firmado.
- No hay bloqueo de ficheros entre procesos concurrentes.
- Los metadatos (usuario, número de secuencia, fecha, número de entradas) no están cifrados.
- No hay recuperación de contraseña: la KEK se deriva de ella, así que perderla implica perder el planning.

Capturas sugeridas: resultado de `python -m unittest discover -s tests -t .` y la demo con los tres ataques.

7. Librerías, versiones e instalación

Dependencias criptográficas. `cryptography==50.0.2` (fijada en `requirements.txt`). Todo lo demás es biblioteca estándar de Python (`unittest`, `argparse`, `getpass`, `json`, `logging`). Probado con Python 3.12.3.

Instalación y ejecución desde un entorno limpio.

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python -m unittest discover -s tests -t .
bash scripts/demo.sh
```

El repositorio no contiene claves privadas, contraseñas ni datos personales: la secretaría y los datos se generan al ejecutar y `.gitignore` excluye `data/` y los ficheros `.pem`.

Anexos (no cuentan para el límite)

- Código fuente completo o enlace al repositorio.
- Salida íntegra de la demo.
- Cualquier captura adicional.

Preparación de la defensa (borrar antes de entregar)

La defensa vale 20 de 30 puntos y puede incluir preguntas individuales. Asegúrate de poder explicar, sin mirar el código:

- Por qué una sola ejecución de scrypt más HKDF en lugar de dos hashes de la misma contraseña.
- Qué es el AAD y qué pasaría sin él.
- Por qué los nonces aleatorios de 96 bits son seguros aquí y cuándo dejarían de serlo.
- Por qué se firma el sobre entero y no solo el cifrado.
- Qué impide que un atacante sustituya la clave pública de un estudiante.
- Qué ocurre si se borra el último registro de notas y cómo se arreglaría.